Università degli Studi Federico II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione

UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Final Project

SPIDERBOT

Authors
Daniele Palomba (P3800316)
Giulio Vestri (P38000317)

Supervisors
Prof. Fabio Ruggiero

June 25, 2026

Contents

1 Introduction

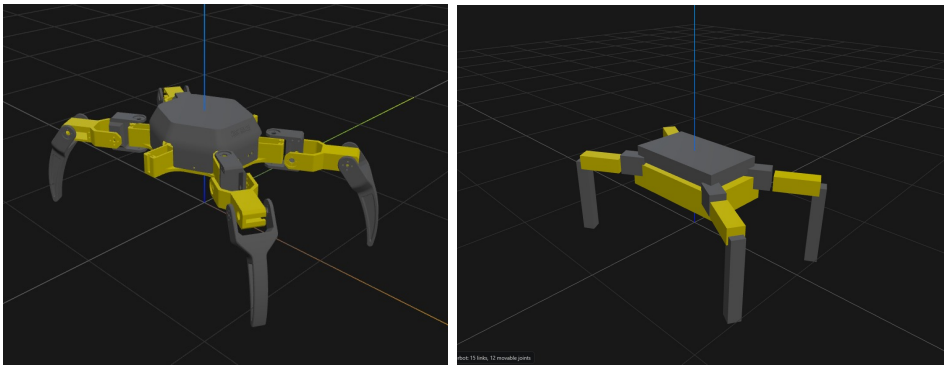
The idea of realizing a project that regards a **spider-like robot** structure came from personal curiosity and satisfaction. But this does not rule out the possibility of deploying such robots in dangerous and hazardous environments where man cannot have access. The real challenge comes from applying robust controllers that ensure stability to the main body and a coordinated motion between the four legs of the robot. By it being a very complex task, in this paper we provide a starting point for our *SpiderBot* by applying an **open-loop system** both on simulation and on **hardware**, and a **end-to-end learning** method was also tested on our experimental structure. This can be an optimal launching pad for future improvements and innovative control applications.

1.1 Robot Model

SpiderBot is a 12-DOF spider-like robot with four legs, comprised of three actuated revolute joints per leg. The original project that can be found **here** is characterized by six legs, in our case the structure was simplified to lower costs and complexity. Given the bio-inspired idea of our structure we defined a series of key components regarding its body parts: the main body of hexagonal form, the coxa, the femur and tibia links. All the parts were designed starting from a CAD model. A **URDF** model was also realized and a **Simulink** model was obtained from the latter as to make sure that its dynamics and kinematics were as accurate as possible to the real structure.

Figure 1.1: *SpiderBot* model

Then from the transformation and connections obtained from the original URDF an **XML** file, comprised of primitive geometric shapes, was used to train the robot inside **MuJoCo Jax** environment.



(a) URDF model (b) XML model

Table 1.1: Physical parameters of the spider robot

Quantity	**Symbol**	**Value**	**Unit**
Total Mass	m_{tot}	1.7	kg
Total Body Inertia (Diagonal)	I_{body}	$[0.004, 0.004, 0.008]$	kg·m^2
Structure Width (Shoulder-to-Shoulder)	W_{body}	0.125	m
Nominal Stance Height	H_{nom}	0.100	m
Coxa Length	L_{coxa}	0.038	m
Femur Length	L_{femur}	0.086	m
Tibia Length	L_{tibia}	0.160	m

Table 1.2: Actuator specifications (MZ996 Servos)

Quantity	**Symbol**	**Value**	**Unit**
Total Number of Motors	N_{motors}	12	-
Maximum Torque	τ_{max}	10.0	N·m
Maximum Speed	ω_{max}	3.0	rad/s

2 Open-Loop Kinematic Control

In this section we present a hierarchical **Open-Loop Kinematic Control Architecture**. The system operates on three distinct layers: a high-level time-driven trajectory generator (Gait Planner), a mid-level spatial transformation layer (Inverse Kinematics), and a low-level decentralized actuation layer (Servo PID). Because the high-level planner does not receive exteroceptive or proprioceptive feedback, the locomotion is strictly open-loop, relying on the inherent static stability of the robot's sprawling morphology and predefined gait profiles to navigate the environment.

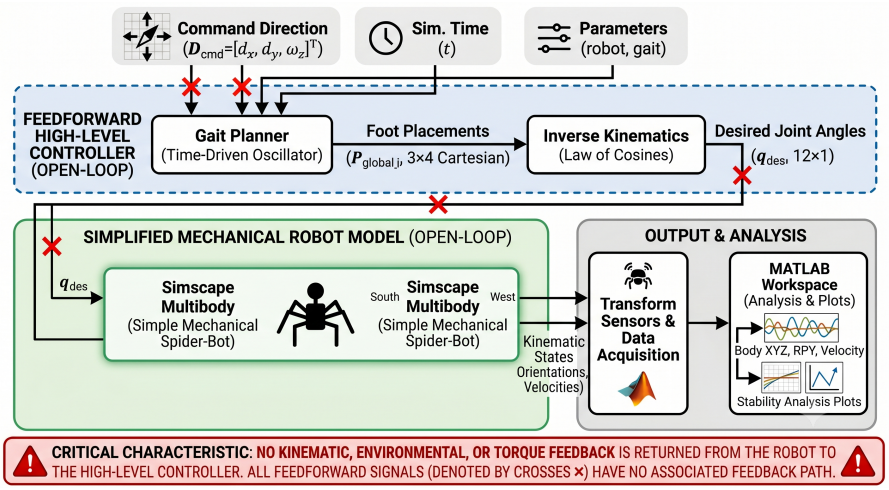


Figure 2.1: Open Loop Block Scheme

2.1 Gait Generator

At the core of the locomotion system is the **Gait Generator**, which computes the desired 3D Cartesian coordinates of each foot (P_{global}) relative to the robot's reference frame. The planner is driven by an absolute simulation clock (t) and a master direction command ($V_{cmd} = [d_x, d_y, \omega_z]^T$). To schedule the sequence of leg movements, the absolute time is converted into a normalized, continuously looping global phase variable $\phi \in [0, 1)$ based on the gait's cycle duration (T):

$$\phi = \left(\frac{t}{T}\right) - \left\lfloor \frac{t}{T} \right\rfloor \tag{2.1}$$

To orchestrate specific multi-beat gaits, each leg i (where $i \in \{1, 2, 3, 4\}$ corresponding to leg order RR, RL, FR, FL) is assigned a distinct phase offset δ_i. The local phase ϕ_i of each leg is computed as:

$$\phi_i = (\phi + \delta_i) \tag{2.2}$$

The local phase dictates whether the leg is in the Swing Phase (moving through the air) or the Stance Phase (pushing against the ground), governed by the duty factor β (the percentage of the cycle spent on the ground). To mitigate impact shocks during high-speed locomotion, the planner employs Velocity Matching. Rather than moving the swing foot linearly, the trajectory is mathematically skewed. During the swing phase, the normalized progression variable s_{swing} is warped using a trigonometric function:

$$s_{skewed} = s_{swing} - k \sin(2\pi s_{swing}) \tag{2.3}$$

where k is a tuning constant (e.g., 0.15). This generates a "teardrop" spatial profile, ensuring that the foot decelerates its forward swing and begins sweeping backward just before touchdown, forcing the relative velocity between the foot and the ground to approach zero at impact.

2.2 Implemented Locomotion Gaits

The open-loop trajectory generator is designed to be highly versatile. By altering the duty factor (β), the phase offsets (δ_i), and applying specific biomechanical spatial transformations, the same kinematic architecture produces four distinct locomotion profiles. These range from a highly conservative, statically stable walk to aggressive, dynamic bounding. The plots show the behaviour of the SpiderBot given various gait modalities with a master direction command of 1.0 on x. The gallop gait also showed best results given a lateral command instead, whereas with the forward command it showed no accurate motion given the difference between parasagittal and sprawling morphologies. The analysis covered the CoM evolution in position, velocity and rotation. The position plots show how the robot correctly follows a smooth, monotonic climb along its commanded trajectory while oscillating in a lateral drift. This represents the robot naturally swaying side-to-side as it shifts its Center of Mass (CoM) to maintain the 3-leg support polygon. Because the drift is bounded and does not compound, the gait is directionally stable, while it also settles from the CAD origin to its nominal stance height of -0.12 m (in NED configuration)h along the entire path. In the angular dynamics we can see important differences between the Creep Gait and the other. In the creep gait we have a statically stable system by looking at the Roll and Pitch angles (Orange and Yellow lines in Fig.2.4). The other gaits show continuous angular oscillations instead, where the robot violently rocks side-to-side during every step cycle. While these rocking oscillations are severe, they are strictly bounded. The amplitudes do not diverge over the 30-second run. The robot successfully catches itself every cycle without flipping over, proving that the choosing of the β duty factor overlap successfully prevents catastrophic roll failure in all gaits.

2.2.1 Mathematical Formulation

Before analyzing the individual locomotion profiles, it is necessary to define the core mathematical engine that generates the spatial trajectories. The gait planner operates as a time-driven, parametric spatial oscillator. It accepts high-level directional commands (d_x, d_y, ω_z) and a set of predefined kinematic constraints, outputting the continuous 3D Cartesian target coordinates for all four feet.

- Cycle Duration (T): The absolute simulation time t is normalized by T to create a continuously looping global phase variable $\phi \in [0, 1)$. This phase acts as the master clock for all legs.
- Duty Factor (β): This parameter dictates the percentage of the cycle a leg spends in contact with the ground. It mathematically bifurcates the local phase ϕ_i of each leg into two distinct continuous domains: the Swing Phase (normalized progression from $0 \to 1$ through the air) and the Stance Phase (normalized progression from $0 \to 1$ against the ground).
- Step Length (L_{step}) Maximum Rotation (R_{max}): These parameters define the spatial amplitude of the stride. During the swing phase, the foot translates linearly from $-L_{step}/2$ to $+L_{step}/2$ along the commanded XY velocity vector, and rotates from $-R_{max}/2$ to $+R_{max}/2$ for yaw steering. During the stance phase, this motion is perfectly inverted to propel the chassis forward.
- Step Height (H_{step}) Stance Depth (Z_{depth}): These govern the Z-axis (vertical) trajectory. During swing, the foot follows a sinusoidal or bell-curve profile peaking at H_{step} to clear obstacles. During stance, the foot is driven to a constant negative coordinate defined by Z_{depth} (e.g., -0.005 m) to ensure positive ground penetration and traction within the physics engine's collision model.

The ultimate output of the planner is $P_{globali}$, a 3×4 matrix containing the absolute (x, y, z) target coordinates for each leg $i \in \{1, 2, 3, 4\}$ relative to the center of the robot's chassis. First, the local foot resting position (`home_locale`) is rotated by the shoulder's mounting angle (θ_i) and added to the shoulder's base coordinates (`spalle_x`, `spalle_y`). This establishes the neutral anchor point $(x_{base}, y_{base}, z_{base})$ for the leg in the global chassis frame.
Based on whether the leg is in the swing or stance phase, the temporal variables are mapped to spatial displacements $(\Delta x, \Delta y, \Delta z)$ and a local steering rotation angle $(\Delta\psi)$. To allow the robot to walk in curves (Yaw steering command $\omega_z \neq 0$), a 2D rotation matrix is applied to the neutral anchor coordinates based on the instantaneous steering angle $\Delta\psi$. Finally, the spatial step displacements are added to yield the final global coordinate for leg i.Mathematically, the i-th column of the $P_{globali}$ matrix is computed

as:

$$\begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix}_i = \begin{bmatrix} \cos(\Delta\psi) & -\sin(\Delta\psi) & 0 \\ \sin(\Delta\psi) & \cos(\Delta\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{base} \\ y_{base} \\ z_{base} \end{bmatrix}_i + \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta z \end{bmatrix}_i$$

This matrix represents the instantaneous target for the feet. It is updated at every simulation step and continuously fed into the Inverse Kinematics solver to drive the physical actuators.

2.2.2 Creep Gait

The Creep is a 4-beat walking gait designed for maximum stability on uneven terrain. It utilizes a high duty factor of $\beta = 0.75$, meaning each leg spends 75% of the cycle on the ground and only 25% in the swing phase. Because of the phase offsets ([0.50, 0.00, 0.75, 0.25] for RR, RL, FR, FL), the robot lifts only one leg at a time. This guarantees that a wide, triangular three-point support polygon is always maintained beneath the Center of Mass (CoM).
To ensure traction in the physics simulation, the stance depth is set to penetrate the ground slightly ($z = -0.005$ m). Additionally, a dynamic height compensation algorithm is applied: if the lateral velocity command (v_y) exceeds 0.5 m/s, the step height is increased by 50% to prevent the sprawling legs from tripping over themselves during sideways strafing.

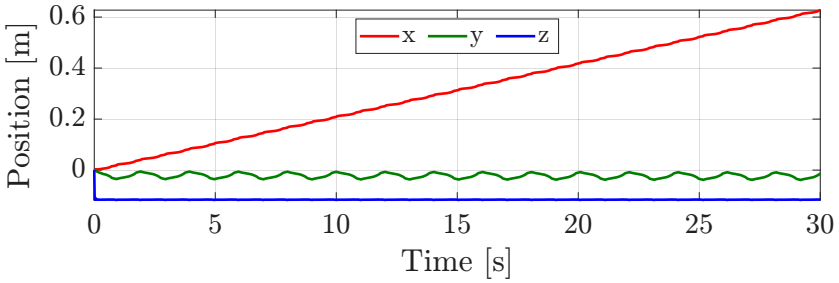


Figure 2.2: Creep CoM Position

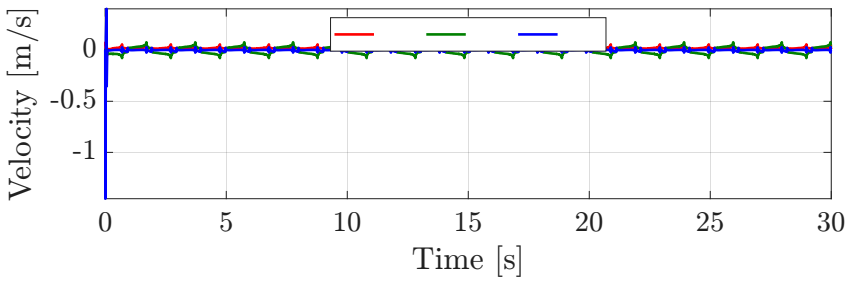


Figure 2.3: Creep CoM Velocity

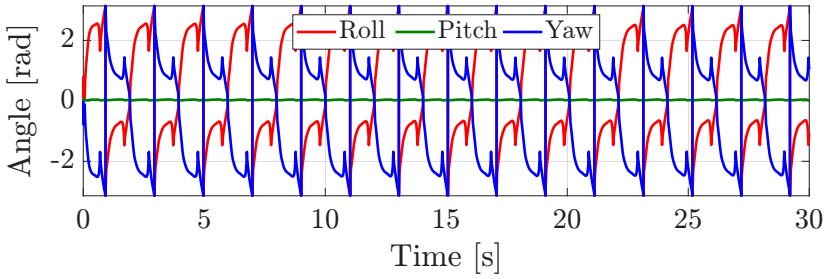


Figure 2.4: Creep CoM Rotation

2.2.3 Trot Gait

The Trot is a medium-to-high-speed 2-beat gait. It operates at a $\beta = 0.50$ duty factor, dividing the cycle equally between stance and swing. The legs are synchronized in diagonal pairs to maximize the stability of the sprawling morphology. The Rear-Left (RL) and Front-Right (FR) legs move together ($\delta = 0.50$), alternating with the Rear-Right (RR) and Front-Left (FL) legs ($\delta = 0.00$).
By utilizing diagonal pairs, the support line passes directly through the geometric center of the robot. As the robot shifts its weight from one diagonal to the other, the CoM requires minimal displacement, effectively eliminating the roll and pitch instabilities that typically cause sprawling robots to flip over at high speeds.

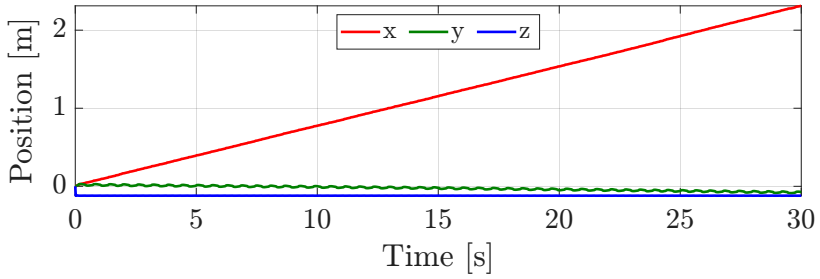


Figure 2.5: Trot CoM Position

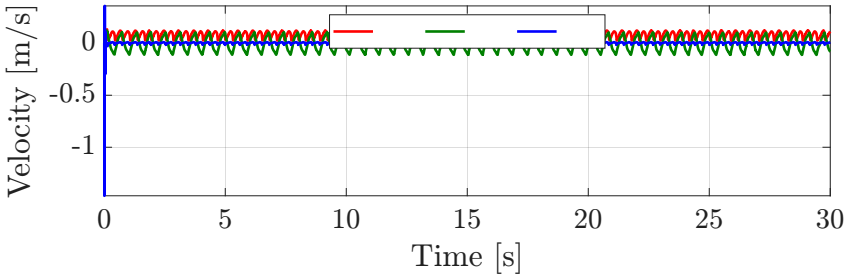


Figure 2.6: Trot CoM Velocity

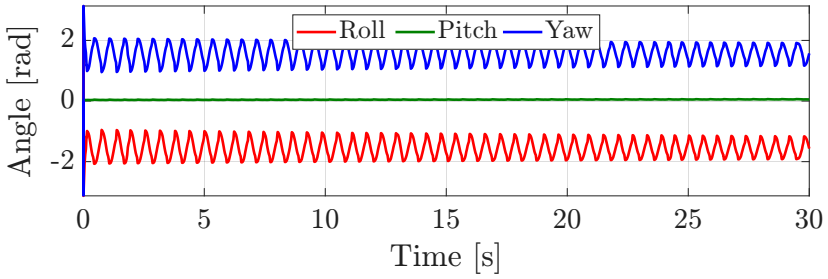


Figure 2.7: Trot CoM Rotation

2.2.4 Gallop Gait

The Gallop is a highly dynamic 4-beat bounding gait. To achieve mammalian-like bounding on a rigid spider morphology, the duty factor is reduced to $\beta = 0.40$ (allowing

for flight phases), and three major biomechanical upgrades are mathematically injected into the trajectory:

1. Asymmetric Leg Roles: The legs are decoupled into two distinct functional groups. The rear legs act as the "engine," utilizing a longer step length (1.3×) and a deeper ground penetration ($z = -0.008$ m) to provide forward thrust. The front legs act as "shock absorbers," utilizing a shorter stride (0.8×) and higher clearance (1.2×) to catch the chassis after the flight phase.

2. The Virtual Spine (Dynamic Pitch): Because the robot's URDF lacks a flexible spine, a continuous sinusoidal pitch oscillation is applied to the global shoulder coordinates:
$$\theta_{pitch} = 0.14 \sin(2\pi\phi_{global}) \tag{2.4}$$
This artificially pitches the nose of the robot up during the front swing phase and down during the rear swing phase, massively increasing the effective stride length.

3. Velocity Matching: A skewed progression curve is applied to the swing trajectory to ensure the foot is already moving backward at the moment of touchdown, eliminating braking shocks.

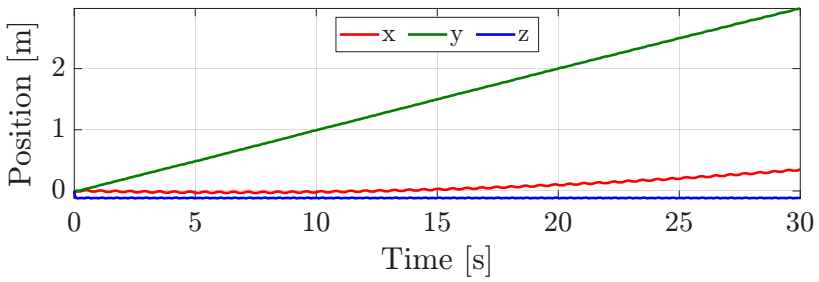


Figure 2.8: Gallop CoM Position

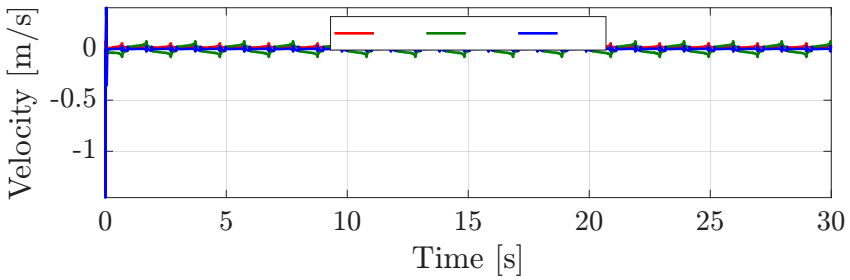


Figure 2.9: Gallop CoM Velocity

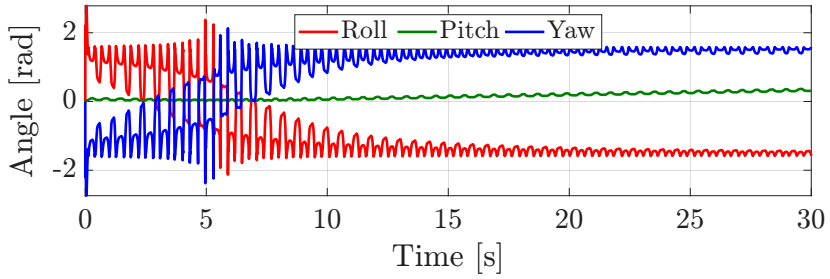


Figure 2.10: Gallop CoM Rotation

2.2.5 Pace Gait

The Pace is a lateral 2-beat gait where the legs on the same side of the body (e.g., RR and FR) move simultaneously. While a pure pace is inherently unstable for a sprawling robot due to severe roll moments, the implemented version introduces advanced compensations to maintain stability.
To prevent the robot from rolling over when a lateral pair lifts, a continuous sinusoidal sway is injected into the Y-axis of the target coordinates:

$$y_{sway} = 0.015 \sin(2\pi\phi_{global}) \tag{2.5}$$

This forces the IK solver to actively shift the robot's chassis 1.5 cm laterally over the grounded legs before lifting the opposite side, mimicking the Zero-Moment Point (ZMP) balancing of pacing animals.
The duty factor is slightly elevated to $\beta = 0.55$. This creates a brief 4-leg stance overlap phase during weight transitions, converting the strict pace into a stable "Running Walk" or Amble.

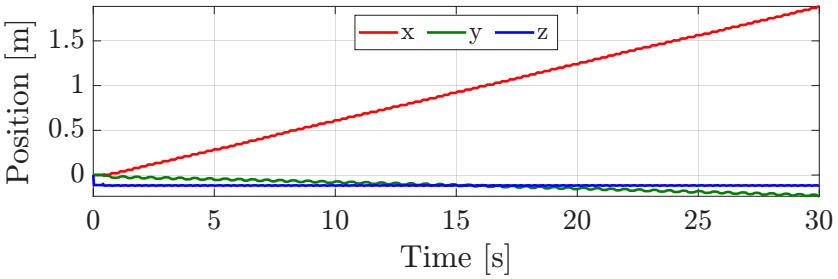


Figure 2.11: Pacing CoM Position

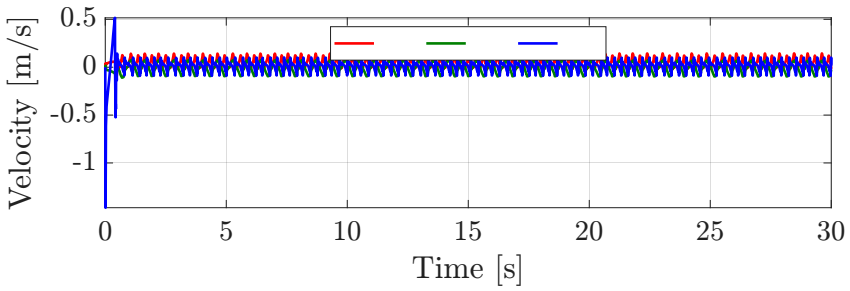


Figure 2.12: Pacing CoM Velocity

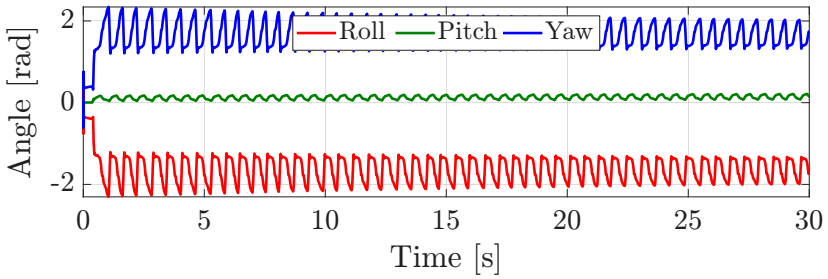


Figure 2.13: Pacing CoM Rotation

2.3 Inverse Kinematics Mapping

The output of the Gait Planner is an array of desired Cartesian foot coordinates. To translate these targets into physical actuator commands, an analytical Inverse Kinematics (IK) solver is implemented for the sprawling 3-Degree-of-Freedom (DoF) legs.
First, the global Cartesian target vector is translated and rotated into the local 2D working plane of the specific leg. Let $(x_{loc}, y_{loc}, z_{loc})$ be the target coordinates relative to the leg's shoulder joint, accounting for the mounting yaw angle θ_i of the Coxa. The Coxa joint angle q_1 is calculated via standard planar trigonometry:

$$q_1 = \text{atan2}(y_{loc}, x_{loc}) \tag{2.6}$$

Once the leg plane is established, the problem simplifies to a 2D planar manipulator solving for the Femur and Tibia angles. The horizontal distance from the Coxa joint to the target in the local XY plane is computed, subtracting the fixed length of the Coxa link (L_c):

$$L_{fwd} = \sqrt{x_{loc}^2 + y_{loc}^2} - L_c \tag{2.7}$$

The direct spatial distance (hypotenuse) D from the Femur joint to the foot target is then calculated:

$$D = \sqrt{L_{fwd}^2 + z_{loc}^2} \tag{2.8}$$

Using the Law of Cosines on the triangle formed by the Femur link (L_f), the Tibia link (L_t), and the hypotenuse D, the Tibia joint angle q_3 (the "knee") is determined:

$$q_3 = \arccos\left(\frac{D^2 - L_f^2 - L_t^2}{2L_f L_t}\right) \tag{2.9}$$

Finally, the Femur joint angle q_2 (the "shoulder pitch") is composed of two sub-angles: the angle to the target point α, and the inner angle of the triangle γ, yielding:

$$q_2 = \text{atan2}(z_{loc}, L_{fwd}) + \text{atan2}(L_t \sin(q_3), L_f + L_t \cos(q_3)) \tag{2.10}$$

These resulting mathematical angles are then adjusted by calibration constants specific to the URDF mounting orientations (e.g., subtracting $\pi/2$ or π to match the zero-positions of the 3D model) before being sent as reference commands q_{des} to the servos.

3 Deep Reinforcement Learning Architecture

3.1 The End-to-End Control Paradigm

Unlike the previously implemented hierarchical control system (where a high-level Gait Planner generated 3D Cartesian paths and an Inverse Kinematics solver mapped them to joint angles), this architecture employs End-to-End Deep Reinforcement Learning (DRL).
In this paradigm, a deep neural network acts as the singular, unified controller. It bypasses geometric trajectory planning entirely, learning a direct mapping from the robot's proprioceptive sensory inputs to raw joint-level actuation commands. The problem is framed mathematically as a Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where the agent interacts with the MuJoCo/Brax physics engine to maximize its expected cumulative reward.
The software architecture was optimized for execution on AMD hardware. This granted us the possibility of achieving high training speed without using GPU-dependent environments like NVIDIA Isaac-Lab.
The process is comprised of four python scripts:

- `spider_env.py`: defines simulation environment, the observation space and reward logic;
- `train_spider.py`: handles the learning algorithm (PPO) maximizing hardware efficiency;
- `export_json.py`: extracts the weights from the trained neural network (in .pkl format) and converts them into universal JSON matrices;
- `enjoy_spider.py`: executes the live simulation, interfacing the neural network with the commands provided and using the observation space to evaluate and visualize robot locomotion.

The following is a theoretical and functional description on how the neural network trains the spiderbot.

3.2 Observation Space (The State $\mathcal{S}$)

The observation space represents the sensory information available to the neural network at each timestep t. For this environment, the state vector $s_t \in \mathbb{R}^{35}$ relies purely on simulated proprioception. The 35-dimensional state vector is constructed by concatenating:

- Positions ($q_{pos} \in \mathbb{R}^{17}$): The angular positions of the 12 leg joints, the Z-height of the chassis, and a quaternion representing the chassis orientation (roll, pitch, yaw). The absolute global X and Y coordinates are deliberately excluded. This forces the policy to be translation-invariant (the robot learns how to walk, rather than memorizing a specific location on the map).
- Velocities ($q_{vel} \in \mathbb{R}^{18}$): The linear and angular velocities of the main chassis, alongside the angular velocities of the 12 individual servos.

To prevent the neural network from exploiting "perfect" simulation data and to ensure robustness for future hardware deployment, Gaussian noise is injected into both the initialization sequence and the continuous observation loop:

$$s_{t,noisy} = s_t + \mathcal{N}(0, 0.003^2) \tag{3.1}$$

3.3 Action Space (The Policy Output $\mathcal{A}$)

The action space defines how the neural network interacts with the physical world. The policy outputs a continuous action vector $a_t \in \mathbb{R}^{12}$, corresponding to the desired target angles for the 12 leg joints. Because neural networks typically output unbounded values, the raw action vector is passed through a hyperbolic tangent activation function and scaled to match the physical radian limits of the joints:

$$u_t = 1.5 \tanh(a_t) \tag{3.2}$$

where u_t represents the commanded joint angles bounded strictly within $[-1.5, 1.5]$ radians.
To prevent the agent from learning highly fragile, simulation-specific kinematics that would fail on a physical robot, actuation noise is injected directly into the control signal. This forces the policy to learn a robust, fault-tolerant gait:

$$u_{t,applied} = \text{clip}(u_t + \mathcal{N}(0, 0.02^2), -1.5, 1.5) \tag{3.3}$$

3.4 The Reward Function ($\mathcal{R}$)

The reward function is the most critical component of the RL environment, as it serves as the sole guiding metric for the optimization algorithm. The current reward shaping is

formulated as a weighted sum of primary objectives and regularization penalties.
The total reward R_t calculated at each timestep is defined as:

$$R_t = R_y - R_x + P_{fall} - P_{posture} - P_{limits} - P_{smooth} \tag{3.4}$$

Instead of incentivizing unbounded speed, the primary objective strictly enforces target velocity tracking along the lateral axis while penalizing orthogonal drift. The primary reward utilizes a squared-exponential (Gaussian) function to incentivize the robot to maintain a precise target velocity ($v_{ref} = 1.0$ m/s) along the Y-axis:

$$R_y = 1.2 \exp\left(-\frac{(v_y - v_{ref})^2}{\sigma_{lin}}\right)$$

where $\sigma_{lin} = 0.5$. This formulation is mathematically superior to linear rewards because it bounds the maximum possible reward at 1.2. If the robot moves too slowly or too fast, the reward decays exponentially. This completely eliminates the "runaway sprint" exploitation commonly seen in RL, forcing a controlled, steady-state amble.
To enforce pure lateral movement, orthogonal drift along the X-axis is actively punished:

$$R_x = 0.3|v_x|$$

This symmetric penalty allows the optimizer to make minor micro-adjustments for balance but heavily suppresses continuous forward walking.

- Catastrophic Fall Penalty (P_{fall}): A sparse, binary boundary condition evaluated against the chassis Z-height (z) and the quaternion tilt sum (t_{tilt}):

$$P_{fall} = \begin{cases} -100.0 & \text{if } z < 0.10 \vee z > 0.50 \vee t_{tilt} > 0.30 \\ 0.0 & \text{otherwise} \end{cases}$$

- Aggressive Posture Penalty ($P_{posture}$): To force an energy-efficient, natural standing pose, a massive Mean Squared Error penalty (weight = 1.0) is applied to the raw joint angles q_i:

$$P_{posture} = 1.0 \left(\frac{1}{12}\sum_{i=1}^{12} q_i^2\right)$$

  By raising this coefficient to 1.0, the network is subjected to extreme pressure to keep the legs tucked closely to the zero-position, actively preventing the wide, splayed stances that strain physical servo motors.

- Mechanical Safety Limits (P_{limits}): A soft-barrier penalty protects the servos from their absolute mechanical limits ($L_{safe} = 0.7$ rad):

$$P_{limits} = 0.5 \sum_{i=1}^{12} \left(\max(0, |q_i| - 0.7)\right)^2$$

  This piecewise function returns strictly 0 inside the safe envelope, but grows exponentially if the agent attempts an excessively long leg stroke.

- Strict Actuator Smoothness (P_{smooth}): To prevent infinite-jerk commands that would strip the joint gears, the first-order derivative of the action space is heavily penalized (weight = 0.8):

$$P_{smooth} = 0.8 \sum_{i=1}^{12} (a_{t,i} - a_{t-1,i})^2$$

By punishing the squared difference between the current action a_t and the previous action a_{t-1} with such a high coefficient, the network is mathematically coerced into outputting highly continuous, sinusoidal-like control signals, mirroring the smooth trajectories of the open-loop kinematic planners.

3.5 Simulation Results

Following an extensive number of training iterations, the achieved results are presented below. The final learned policy is initially saved as a `.pkl` file and subsequently exported to `JSON` format to be processed by the simulation and control system managed by the `enjoy_spider.py` script. The control architecture implemented within this script follows a standard neural network-based closed-loop framework: the robot provides state observations (`obs`) as inputs to the neural network, which in turn computes and outputs the control actions (`action`) to be applied to the robot.

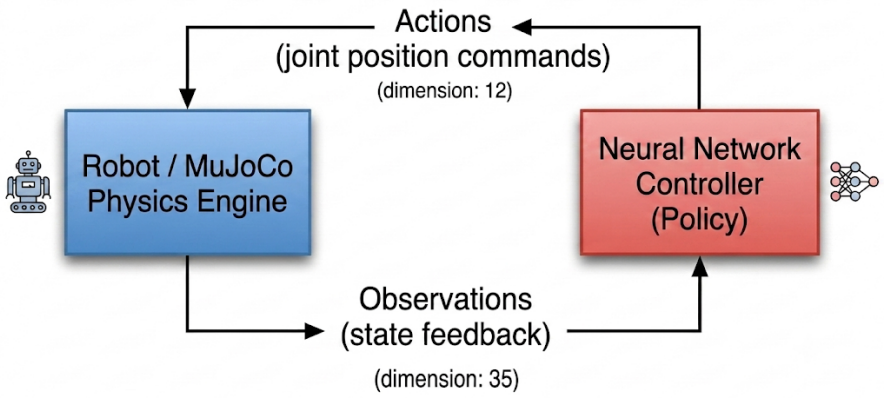


Figure 3.1: Block Diagram of The Neural Network control

Crucially, the resulting behavior directly reflects the mathematical realities of the programmed reward function ($R_t = v_y - 0.3|v_x|$), proving that the agent learned to maximize lateral strafing while actively suppressing forward movement.

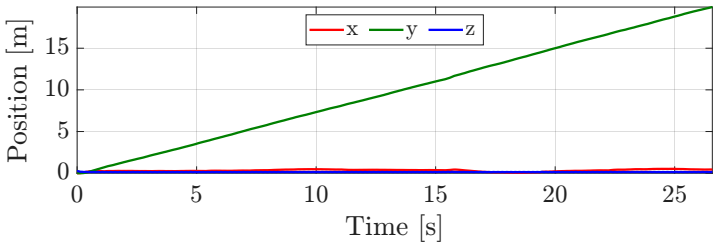


Figure 3.2: Position Behaviour

The spatial tracking plot in Fig.3.2 provides definitive proof of the agent's learned objective. The y-axis position exhibits a perfectly steady, monotonic climb, carrying the robot to nearly 20 meters over the 26-second horizon. The x-axis position remains virtually flat at 0 meters for the entire episode. The Neural Network recognized the $-0.3|v_x|$ penalty and learned a gait that entirely eliminates forward drift and the z-axis position remains tightly bound near 0 (representing the relative nominal stance height), confirming the robot never triggered the catastrophic fall penalty (P_{fall}).

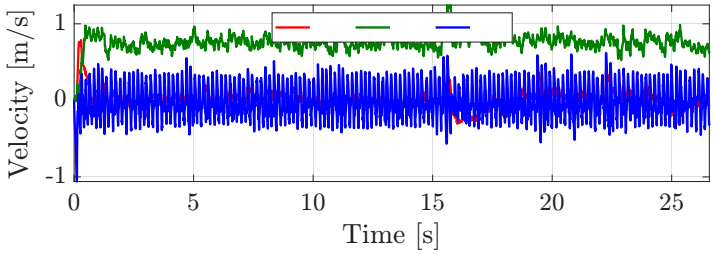


Figure 3.3: Velocity Behaviour

In Fig.3.3 we see the y-velocity (green line) rapidly accelerating from zero and stabilizing in a highly consistent band around 0.8 m/s with a 1.0 m/s velocity reference. This is an exceptionally fast lateral amble for a sprawling robot, achieved entirely through self-play exploration. The x-velocity (red line) is held strictly near zero, further confirming the agent's avoidance of the shaping penalty.
The blue line exhibits massive, zero-mean, high-frequency oscillations. In RL-driven legged locomotion, this is a classic signature of the "stepping frequency." Because the neural network controls the raw joint angles directly without a smoothing trajectory planner, the resulting footfalls create continuous, high-impact accelerations on the chassis.

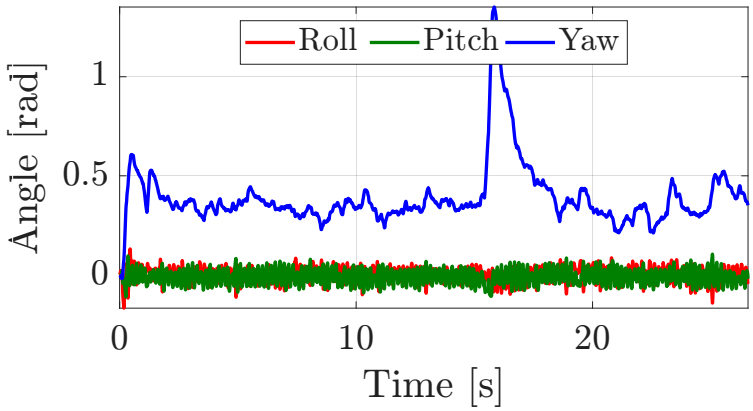


Figure 3.4: Rotation Behaviour

The plot in Fig.3.4 tracks the roll, pitch, and yaw of the chassis, offering excellent insight into the policy's dynamic balancing capabilities.
The Roll (red) and Pitch (green) angles are exceptionally stable, bounded tightly around 0 radians. This proves that the regularization penalty ($P_{posture}$) successfully taught the agent to keep its Center of Mass strictly within the support polygon. The Yaw angle (blue) shows a tendency to drift into a slight crab-walk orientation (stabilizing near 0.4 radians). Most notably, at approximately 15 seconds, the robot experiences a severe disturbance, causing the yaw to spike violently past 1.0 radian.
Despite this massive disturbance at $t = 15$s, the Neural Network policy does not fail. It successfully reacts to the unexpected rotation, stabilizes the chassis, and drives the yaw back down to its nominal operating band by $t = 20$s. This moment of recovery definitively highlights the primary advantage of closed-loop Deep Reinforcement Learning over open-loop kinematic planners: robustness to unexpected physical perturbations.

3.6 Goal-Conditioned Locomotion and Joystick Teleoperation

The initial implementation of the quadrupedal locomotion policy via Reinforcement Learning (RL), while yielding stable dynamic behavior and acceptable baseline performance, presented severe functional limitations regarding advanced robotic control. Specifically, the agent was trained exclusively to track a static forward velocity reference along the longitudinal axis (y). Consequently, the robot was completely incapable of moving, steering, or generalizing its gait in any other spatial direction.

To enable real-time teleoperation via a standard USB joystick—thereby achieving continuous, fluid, and omni-directional motion within the simulation space—the framework was transitioned into a goal-conditioned RL paradigm. This shift required a profound restructuring of both the simulation environment and the underlying software pipeline, executed through four foundational modifications:

- **State Space Integration (Observation Space):** The dimension of the neural network's input vector was increased from 35 to 38. This expansion explicitly integrates the 3 instantaneous kinematic command targets provided by the joystick (v_x, v_y, and the angular yaw velocity ω_z).
- **Reward Shaping and Extended Curriculum:** The optimization objective was reformulated to operate over multi-dimensional velocity coordinates. The new reward:

$$R_{\text{total}} = R_{\text{survival}} + R_{\text{tracking}} + P_{\text{fall}} - P_{\text{drift}} - P_{\text{posture}} - P_{\text{smooth}} - P_{\text{energy}} \tag{3.5}$$

  Where the individual terms are defined as:

$$R_{\text{survival}} = 1.0 \tag{3.6}$$

$$R_{\text{tracking}} = 3.0 \cdot \exp\left(-\frac{(v_x - c_{vx})^2 + (v_y - c_{vy})^2 + (\omega_z - c_{wz})^2}{\sigma_{\text{vel}}}\right) \quad \text{with } \sigma_{\text{vel}} = 0.25 \tag{3.7}$$

$$P_{\text{drift}} = 2.0 \cdot \sum_{i \in \{x,y,wz\}} \exp\left(-\frac{c_i^2}{\epsilon}\right) \cdot v_i^2 \quad \text{with } \epsilon = 0.01 \tag{3.8}$$

$$P_{\text{fall}} = \begin{cases} -100.0 & \text{if } z < 0.1 \vee z > 0.50 \vee (\phi^2 + \theta^2) > 0.3 \\ 0.0 & \text{otherwise} \end{cases} \tag{3.9}$$

$$P_{\text{posture}} = \frac{0.5}{12} \sum_{j=1}^{12} q_j^2 \tag{3.10}$$

$$P_{\text{smooth}} = 0.5 \cdot \sum_{k=1}^{12} (a_{t,k} - a_{t-1,k})^2 \tag{3.11}$$

$$P_{\text{energy}} = 0.02 \cdot \sum_{k=1}^{12} u_k^2 \tag{3.12}$$

  The function actively minimizes the tracking error between the actual chassis linear/angular velocities and the dynamic reference inputs generated by the user. Furthermore, an *Extended Curriculum* was introduced to impose severe penalties for kinematic drift, explicitly punishing movements along uncommanded degrees of freedom to ensure clean, decoupled omni-directional locomotion.

- **Pipeline Synchronization:** The training (`train_spider.py`), interactive inference (`enjoy_spider.py`), and parameter exportation (`export_json.py`) scripts were refactored to ensure tensor dimension consistency.

Additionally, we implemented a function that generates a CSV file containing the joystick teleoperation data and the actual velocities developed by the robot during the simulation. The following are the resulting plots from 5 continuous days of training:

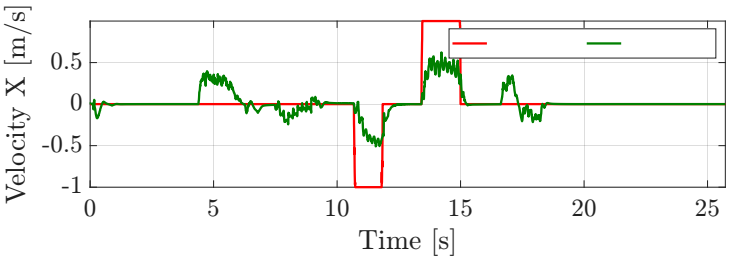


Figure 3.5: Tracking of v_x

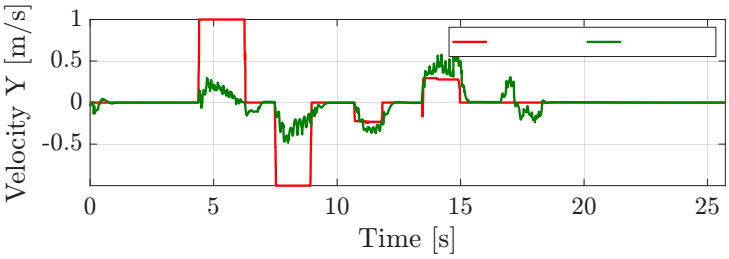


Figure 3.6: Tracking of v_y

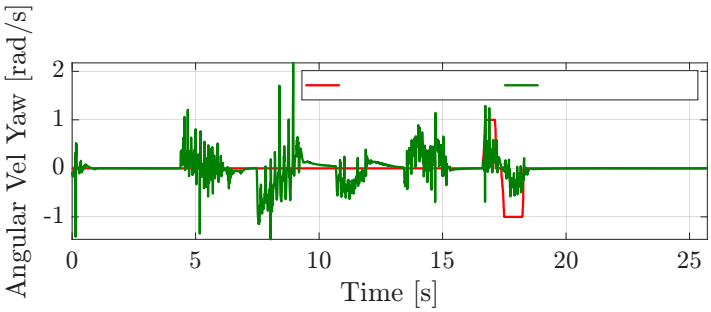


Figure 3.7: Tracking of yaw

4 Hardware Implementation

This chapter details the physical architecture, electronic configuration, and firmware implementation of the *SpiderBot* quadruped. The physical prototype is entirely driven by the **Open-Loop Kinematic Control** architecture described in Chapter 2. The onboard microcontroller autonomously handles the hierarchical gait generation and Inverse Kinematics (IK), while an external host computer acts purely as a teleoperation interface, streaming high-level directional commands.

4.1 Electronic Architecture and Control Flow

The system employs a master-slave topology. A central host machine (a PC running MATLAB) acts as the master, polling user inputs and transmitting reference velocity commands. The local microcontroller board on the robot acts as the low-level slave, mathematically resolving the required 3D trajectories and driving the physical actuators in real-time.

4.1.1 MATLAB Teleoperation Interface

Directional control is handled by a MATLAB script (`teleop_joystick.m`) that interfaces a standard gamepad (via the `vrjoystick` function) with the PC's serial port. The code samples the analog axes at a fixed frequency (20 Hz) and constructs the directional command vector:

$$\mathbf{v}_{\text{cmd}} = \begin{bmatrix} d_x \\ d_y \\ \omega_z \end{bmatrix} \tag{4.1}$$

where d_x and d_y are the directional commands, and ω_z is the yaw rotation rate. The script also monitors button presses to allow the operator to toggle between pre-programmed gaits (e.g., Creep or Trot). These variables are packed into a lightweight 16-byte binary payload (including synchronization headers and checksums) and transmitted via USB-Serial (115200 baud) to the robot.

4.1.2 Local MCU and Firmware (Arduino)

The kinematic processing is fully delegated to the onboard Arduino MCU. The firmware (`SpiderBot.ino`) is the direct C++ realization of the mathematical models presented in the Open-Loop Kinematic Control chapter. At every control cycle, the MCU executes the following deterministic tasks:

1. **Serial Parsing:** Safely receives and decodes the 16-byte command packets from MATLAB.
2. **Gait Planning:** Advances the global phase variable t and computes the target Cartesian coordinates (x, y, z) for all four feet, acting as the spatial oscillator driven by the $\mathbf{v}_{\text{cmd}}$ input.
3. **Inverse Kinematics (IK):** Translates the generated 3D targets into local joint angles $(\theta_1, \theta_2, \theta_3)$ using the planar trigonometric solver.

4.1.3 PCA9685 PWM Driver and Signal Generation

To simultaneously command the 12 servomotors without saturating the Arduino's internal timers—which would cause severe signal jitter and mechanical shaking—the MCU offloads PWM generation to a **PCA9685** expansion module via the I2C bus. The joint angles computed by the IK algorithm (in radians) are linearly mapped into 12-bit PWM pulses (empirically calibrated between 120 and 520 ticks). The hardware pin mapping is structured to mirror the URDF model, as shown in Table 4.1.

Table 4.1: Hardware channel mapping on the PCA9685 PWM driver.

Leg ID	**Joint**	**PCA9685 Pin**
Front Right (`FR`)	Coxa (Yaw)	14
	Femur (Pitch)	13
	Tibia (Pitch)	12
Front Left (`FL`)	Coxa (Yaw)	10
	Femur (Pitch)	9
	Tibia (Pitch)	8
Rear Left (`RL`)	Coxa (Yaw)	2
	Femur (Pitch)	1
	Tibia (Pitch)	0
Rear Right (`RR`)	Coxa (Yaw)	6
	Femur (Pitch)	5
	Tibia (Pitch)	4

4.1.4 Structural Geometry and Components

The *SpiderBot* chassis inherits a hexagonal geometry, providing an exceptionally wide static support base. Mechanical actuation relies on 12 digital **MZ996R** servomotors. The physical link dimensions embedded within the firmware's IK solver perfectly match the 3D-printed components:

- Coxa Length (L_{coxa}): 38.0 mm
- Femur Length (L_{femur}): 86.25 mm
- Tibia Length (L_{tibia}): 160.5 mm

The entire structure was manufactured via Fused Deposition Modeling (FDM) 3D Printing using **PLA** filament. The rotational joints and servo horns were assembled using various lengths of M3 metric screws.
Here is a video demonstration of the hardware application. It is shown how the behaviour of the physical spiderbot changes given a defined gait commanded from a controller input.